

Model Predictive Runtime Verification for Embedded Platforms with Real-Time Deadlines^{*}

Pei Zhang, Rohit Dureja, Matthew Cauwels, Joseph Zambreno,
Phillip H. Jones, Kristin Y. Rozier

Iowa State University, USA

Abstract. Reactive systems require mission-time verification that accurately captures future-time requirements. On-board runtime verification (RV) requires real-time responsiveness that can be embedded within the limited resources (timing, power, weight, bandwidth, hardware, etc.) of the running system. For effective mitigation triggering, RV must detect faults as-early-as-possible, plus make best-possible predictions of future faults on-deadline. Effective on-board RV will always need to know or predict whether, e.g., the parachute needs to be deployed by the deadline for it to be effective (e.g., to cushion the aircraft’s landing). To maximize accuracy and reduce mission-ending mistakes, RV must use real data whenever possible. We contribute a formal definition of real-time Model Predictive Runtime Verification (MPRV) that uniquely uses real data wherever possible, predicting only the minimum variable values required to make an on-deadline, best-as-possible requirement evaluation. MPRV carries a design-time model to runtime to fill in individual predictive values only when necessary. We devise a generic algorithm for MPRV on embedded platforms with limited resources, proving the worst-case execution time and memory usage. We contribute a specialized algorithm for implementing MPRV in FPGA hardware and argue for its correctness. We implement MPRV as a significant extension to the R2U2 RV engine, release a new version of that tool, and conduct an extensive experimental analysis demonstrating its utility and performance.

1 Introduction

Modern safety-critical systems like Unmanned Aerial Systems (UAS), autonomous driving systems, and human-interactive robots require formal verification to be carried throughout the system lifecycle from design time to runtime, to ensure that they uphold design requirements and react as-safely-as-possible to unanticipated events during mission deployment. The nature of requirements for these systems mandates future-time specification; for example, aerospace operational concepts are drawn as future-time timelines, e.g., [32,14,50,39,37]. On-board runtime verification (RV) requires optimized algorithms for real-time responsiveness that can be implemented on the available hardware, e.g., an FPGA or resource-limited flight computer, and embedded within the system’s resource constraints, including unobtrusiveness, timing, power, weight, bandwidth, hardware resources, cost, bus capacity, or other limitations of the running system.

^{*} Work supported by NASA ECF NNX16AR57G and NSF CAREER Award CNS-1664356. Full-version with proofs, hardware implementation, detailed experiments and reproducibility artifacts is available at <http://temporallogic.org/research/TACAS21/>.

RV as an effective mitigation trigger often requires some element of prediction because waiting until there is enough information to confirm a fault has occurred cuts off mitigation options. For example, UAS are launched with a small, finite set of possible mitigation actions should a fault occur and each of those actions (e.g., deploying a parachute) has an a priori-known deployment time. RV needs to trigger the parachute in time to unfurl and cushion the landing, but since such an event would be mission-ending, it is important to be as accurate as possible, minimizing predicted inputs.

Effective mitigation triggering utilizes the known deployment times of each mitigation strategy and triggers as late as possible to maximize confidence. Given the prior example of a mission-ending event (e.g. a UAS motor failure) and mitigation deployment constraint (like deploying a parachute that takes three seconds and must be deployed within two seconds after motor failure to be effective at this altitude). We can define a mitigation trigger *deadline* d , where in this case d is one second before motor failure. Model Predictive Runtime Verification (MPRV) combines the real system data up to time d with the minimal necessary predicted data to make the best-as-possible decision whether to trigger the parachute; this decision must be made at time deadline d , for every second during the mission. Note that d can change with the property being monitored or with the mission. For example, the same mission property may require evaluation with a short d when the UAS is flying at low altitudes, but then with no d at all when the UAS is flying so low to ground (say for ground inspection tasks) that a parachute would not be useful. The same property may require monitoring with a longer d when the UAS is flying high-altitude survey tasks; in this case there may even be time to rigorously confirm the motor failure without any predicted data. MPRV generically encapsulates all of these operational scenarios.

Runtime verification, or evaluating whether some specification φ holds over a system run, has been widely studied, e.g., see [10] for a survey. Real-time mitigation of reactive systems requires *online, stream-based* RV, that is, RV that executes *during system execution time* and operates over a stream of inputs π , producing a stream of output $\langle \text{time}, \text{verdict} \rangle$ tuples evaluating whether φ holds over π starting from time , for all times in the input stream. Embedded RV presents several unique challenges: a) it must be able to run on-board on resource-constrained computers; b) it must be fast enough to return verdicts in real time; c) it must be able to execute unobtrusively without altering the operational capabilities of the system. Importantly, RV engines must be able to encode generic, re-usable specifications that capture the future-time requirements given in operational concepts. **Only two RV engines run in real time on embedded platforms (R2U2 [37] and LOLA [12,2]) and neither of them provide prediction.** Many RV designers have recognized the need for hardware implementations suitable for real embedded systems [35,33,19,44,41,29,20,42] though of these R2U2 is the only one that has a currently-maintained implementation, evaluates both *true* and *false* verdicts (rather than only property failure) over future-time logics without evaluation delay, and monitors formulas (versus analysis requiring software instrumentation). See [16] for a taxonomy encompassing modern RV engines.

Related Work Many have recognized the necessity of prediction in RV, though the context has varied widely. Several works consider prediction of *untimed properties* through static analysis of the system’s code or model checking [26,49,15]. Offline run-

time verification via comparing runs with models has successfully reduced the size and increased the accuracy of model-checking models [51]. Others propose new predictive logics to allow specification over system models with probabilistic or predictive states [26,43], though these do not synthesize monitors that run on-board or in real time. Several MATLAB-based tools focus on learning new information about a system via prediction from system runs [6,5,13,47] such as how a system would behave in new situations or the robustness of the system’s specifications to perturbations. Similarly, TeSSLa has recently been extended to analyze non-synchronized timed event streams for repairing streams with missing data [27], which is a different problem, though it would be worthwhile to investigate cross-pollination of that algorithm with MPRV, e.g., to improve the accuracy of MPRV verdicts that involve prediction. The most closely related work [36] also synthesizes an online monitor for a timed property and utilizes real data for evaluation. However, the output is a three-valued semantics indicating whether it is still possible to satisfy or violate the property; the monitors are synthesized as software-implemented timed-automata, which are not compositional, resource-aware, or executable in real time or in hardware. **None of these tools or algorithms address the MPRV question: given a mitigation trigger time deadline d , determine at time d the most-likely two-valued verdict (true or false) whether to trigger mitigation.** While some previous works could be extended to answer this question in specific contexts (e.g. via static analysis of software), no existing tool answers this question in an implementation-independent (analyzing hardware or software systems without software instrumentation), real-time, stream-based, online setting, much less in a resource-aware, on-board embeddable fashion, such as in hardware (FPGA). Also, no existing predictive tool utilizes maximal real data, rather than relying entirely on predicted data to reach a verdict. Since the deadline d may vary across different properties being monitored on the same embedded system, and in one property across different flight modes of the same embedded system (e.g., a monitor fault detection property when the UAS is flying at higher vs lower altitudes), we require a modular, generalized algorithm for MPRV. Since evaluating property φ at deadline d may require no predication, a mix of real data evaluation and minimal prediction, or a verdict based entirely on prediction, our MPRV definition captures best-as-possible evaluation across these scenarios.

We choose R2U2 as the RV engine for our implementation, largely because it’s formula-wise decomposition of monitors essentially gives us the required formula rewriting capabilities for free and makes the proofs of efficient implementation easier, though we argue that LOLA would have also been a good choice. Either way, an initial MPRV implementation demands a significant extension to hook into any current RV tools. We exemplify the R2U2 extension here and leave the LOLA extension to future work. We choose Mission-time Linear Temporal Logic (MLTL) for property specification as it intuitively captures our UAS case study’s requirements and R2U2 encodes it natively; however, MPRV is extensible to other logics and other RV engines.

Contributions We contribute 1) the formal definition of online MPRV, 2) an algorithm for MPRV on embedded platforms with limited resources with worst-case execution time and memory usage analysis, 3) a hardware implementation of MPRV inside the known R2U2 toolchain, and 4) experimental results demonstrating the performance characteristics. The remainder of this paper is organized as follows. Section 2 provides

the syntax and semantics of Mission-Time Linear Temporal Logic (MLTL) and details the R2U2 interface we utilize for our new implementation. Section 3 formally defines MPRV and gives a general algorithm, including proofs of resource usage and worst-case execution-time bounds. We conduct an extensive experimental analysis in Section 4, evaluating the utility and performance of MPRV utilizing example physics-based models. Section 5 concludes with future directions.

2 Preliminaries

2.1 Mission-Time Linear Temporal Logic (MLTL) [37,28]

Given natural numbers a and b , such that $0 \leq a \leq b$, a *closed interval* is a set of natural numbers $I = \{i \mid a \leq i \leq b\}$, denoted as $I = [a, b]$. The interval I is *bounded* iff $b < +\infty$; otherwise I is *unbounded*. MLTL temporal operators are restricted to bounded intervals. MLTL restricts Metric Temporal Logic (MTL) [4], which includes open and half-open intervals. Note that every MTL open or half-open interval corresponds to an equivalent MLTL closed interval, e.g., $(1, 2) = \emptyset$, $(1, 3) = [2, 2]$, and $(1, 3] = [2, 3]$.

Definition 1. (MLTL Syntax) *The syntax of an MLTL formula φ over a set of atomic propositions $\mathcal{AP}$ is recursively defined as:*

$$\varphi ::= \text{true} \mid \text{false} \mid p \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \Box_I \varphi \mid \Diamond_I \varphi \mid \varphi_1 \mathcal{U}_I \varphi_2 \mid \varphi_1 \mathcal{R}_I \varphi_2$$

where $p \in \mathcal{AP}$ is an atom, and φ_1, φ_2 are MLTL formulas. I is a closed interval $[lb, ub]$, or simply $[ub]$ if $lb = 0$. Given two MLTL formulas φ_1, φ_2 , we denote $\varphi_1 \equiv \varphi_2$ if they are semantically equivalent. In MLTL semantics, we define $\text{false} \equiv \neg\text{true}$, $\varphi_1 \vee \varphi_2 \equiv \neg(\neg\varphi_1 \wedge \neg\varphi_2)$, $\neg(\varphi_1 \mathcal{U}_I \varphi_2) \equiv (\neg\varphi_1 \mathcal{R}_I \neg\varphi_2)$ and $\neg\Diamond_I \varphi \equiv \Box_I \neg\varphi$. MLTL keeps the standard operator equivalences from LTL, including $\Diamond_I \varphi \equiv (\text{true} \mathcal{U}_I \varphi)$ and $(\Box_I \varphi) \equiv (\text{false} \mathcal{R}_I \varphi)$. Notably, MLTL discards the next ($\mathcal{X}$) operator, which is essential in LTL, since $\mathcal{X}\varphi$ is semantically equivalent to $\Box_{[1,1]}\varphi$ (see [28]). The semantics of MLTL formulas are interpreted over bounded finite traces. Let π be a finite trace in which every position $\pi(i)$ ($i \geq 0$) is over $2^{\mathcal{AP}}$.

Definition 2. (MLTL Semantics) *We interpret MLTL formulas over finite computations (traces) π of atomic propositions in $\mathcal{AP}$. We recursively define $\pi, i \models \varphi$ (computation π starting from index $i \geq 0$ satisfies, or “models” MLTL formula φ) as*

- $\pi, i \models \text{true}$,
- $\pi, i \models p$ for $p \in \mathcal{AP}$ iff $p \in \pi(i)$,
- $\pi, i \models \neg\varphi$ iff $\pi, i \not\models \varphi$,
- $\pi, i \models \varphi_1 \wedge \varphi_2$ iff $\pi, i \models \varphi_1$ and $\pi, i \models \varphi_2$,
- $\pi, i \models \varphi_1 \mathcal{U}_{[lb, ub]} \varphi_2$ iff $|\pi| \geq i + lb$ and $\exists j \in [i + lb, i + ub]$ such that $\pi, j \models \varphi_2$ and for every $k < j$ and $k \in [i + lb, i + ub]$ we have $\pi, k \models \varphi_1$,

where $\pi(i)$ is the set of atomic propositions that evaluate to true at index i in trace π , and $|\pi|$ denotes the length of the trace. Note that the semantics of MLTL formulas with $\vee, \Box$, and $\Diamond$ operators can be defined analogously based on operator equivalences. We specify a *trace segment* using bounded closed intervals, i.e., $\pi[m, n]$ is the trace segment bounded between integer indexes m and n .

2.2 Abstract Syntax Tree

A temporal logic formula φ can be evaluated on a trace π in multiple ways [31,38]. R2U2 [40] decomposes the formula into subformulas using a parse tree. It computes

the satisfaction of every subformula from the bottom-up and propagates the verification results to the top-level formula φ . It uses automatic code generation to synthesize subformula logic (asynchronous observers), wherein, the input and output of the synthesized code is in the form of an *execution sequence* over finite time stamps.

Definition 3. (*Execution Sequence for Asynchronous Observers [37]*) An execution sequence for an MLTL formula φ , denoted $\langle T_\varphi \rangle$, over computation π is a sequence of tuples $T_\varphi = (v, \tau)$ where $\tau \in \mathbb{N}_0$ is a time stamp and $v \in \{\text{true}, \text{false}\}$ is a verdict.

We use an integer superscript to access a particular tuple in $\langle T_\varphi \rangle$, e.g., T_φ^0 is the first tuple in $\langle T_\varphi \rangle$. Elements in T_φ are referenced as $T_\varphi.\tau$ and $T_\varphi.v$. We say T_φ holds if $T_\varphi.v$ is true, and T_φ does not hold if $T_\varphi.v$ is false. For a given execution sequence $\langle T_\varphi \rangle = T_\varphi^0, T_\varphi^1, T_\varphi^2, T_\varphi^3, \dots$, the tuple accessed by T_φ^n corresponds to a section of satisfaction of φ such that $\forall i \in [T_\varphi^{n-1}.\tau + 1, T_\varphi^n.\tau]$, we have $\pi, i \models \varphi$ when $T_\varphi^n.v$ is true and $\pi, i \not\models \varphi$ when $T_\varphi^n.v$ is false. We say an execution sequence tuple T_φ is *produced* by the observer for φ when $T_\varphi.\tau \geq i$ and $T_\varphi.v \in \{\text{true}, \text{false}\}$, according to whether $\pi, i \models \varphi$.

Abstract syntax tree construction. A compiler parses the user-specified MLTL formula into an Abstract Syntax Tree (AST) of subformulas, where each node in the AST handles one MLTL operator. Every node explicitly exposes the logical connection between the subformulas. A runtime observer is automatically synthesized for every node in the tree (i.e., for every MLTL operator) that takes input (execution sequence) from child nodes and produces an output (execution sequence) for the parent node. Compiler optimization techniques, such as Common Subexpression Elimination (CSE), can be used to minimize the number of nodes in the AST. The satisfaction of the MLTL formula on an input trace is checked by evaluating the output of the observers from the leaf nodes to the root of the tree. Fig. 1 shows the AST for MLTL formula $\varphi = (\Box_{[0,2]} a0) \wedge a1$ and Fig. 2 shows the corresponding compiled instructions. The leaf nodes are atomic proposition load operators that output an execution sequence that combines the measured value of the atomic propositions in the trace π with time stamps τ . The output sequence of the root node corresponds to the verification result of the MLTL specification φ at every time stamp.

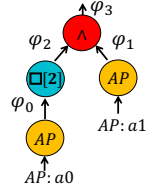


Fig. 1. AST representation

Line 0 : $\varphi_0 \leftarrow \text{load}(a0)$
Line 1 : $\varphi_1 \leftarrow \text{load}(a1)$
Line 2 : $\varphi_2 \leftarrow f_{\Box_{[0,2]}}(\varphi_0)$
Line 3 : $\varphi_3 \leftarrow f_{\wedge}(\varphi_1, \varphi_2)$

Fig. 2. Instruction representation from AST

Abstract syntax tree evaluation. Let t_R denote a real-time metric to describe the current time stamp during runtime. Note that $t_R = 0$ indicates start of execution. Let $\alpha_i(p)$ be the assignment to atom $p \in \mathcal{AP}$ in trace π at time stamp $i \leq t_R$. Table 1 shows the execution sequences generated by the observers when evaluating the MLTL formula $\varphi = (\Box_{[0,2]} a0) \wedge a1$ over the assignments shown in Fig. 3. The atomic proposition load operators capture the hardware signal at the beginning of each time stamp

(shown as event edge in Fig. 3) and output the execution sequence. For example, at time stamp $t_R = 0$ we have $\alpha_0(a1) = \text{true}$ and $T_{\varphi_2} = (\text{true}, 0)$. Similarly, at time stamp $t_R = 3$ we have $\alpha_3(a1) = \text{false}$ and $T_{\varphi_2} = (\text{false}, 3)$. The verdicts in an execution sequence are an exact evaluation of a future-time MLTL formula for each time stamp. They can be utilized to resolve verdicts of other formulas for time stamps $t < t_R$ if new information becomes available at t_R . At time stamp $t_R = 2$ we have $\langle T_{\varphi_0} \rangle = \langle (\text{true}, 0), (\text{true}, 1), (\text{true}, 2) \rangle$ that resolves the verdict of execution sequence T_{φ_1} at time stamp $t = 0$ as $(\text{true}, 0)$. In other words, whether φ_1 holds at time stamp $t = 0$ is resolved only at the current time stamp $t_R = 2$. In the example, the trace π does not satisfy φ because of the tuple $(\text{false}, 3)$ in T_{φ} is produced at time stamp $t_R = 3$.

MLTL \ t_R	0	1	2	3	4
$T_{\varphi_0}(\varphi_0 = \text{load}(a0))$	$(\top, 0)$	$(\top, 1)$	$(\top, 2)$	$(\top, 3)$	$(\top, 4)$
$T_{\varphi_2}(\varphi_2 = \text{load}(a1))$	$(\top, 0)$	$(\top, 1)$	$(\perp, 2)$	$(\perp, 3)$	$(\top, 4)$
$T_{\varphi_1}(\varphi_1 = \Box_{[0,2]}\varphi_0)$	-	-	$(\top, 0)$	$(\top, 1)$	$(\top, 2)$
$T_{\varphi}(\varphi = \varphi_1 \wedge \varphi_2)$	-	-	$(\top, 0)$	$(\top, 1), (\perp, 3)$	-

Table 1. Output of observers as an execution sequence ($\top \equiv \text{true}$ and $\perp \equiv \text{false}$) at time stamps t_R .

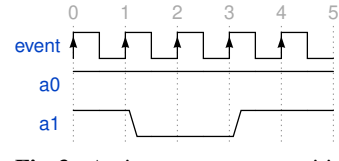


Fig. 3. Assignment to propositions $a0$ and $a1$ at event edges.

3 Model Predictive Runtime Verification (MPRV)

Definition 4. (MLTL Satisfiability) Given an MLTL formula φ , the satisfiability problem asks whether there exists a finite trace π starting at index i , such that $\pi, i \models \varphi$.

To evaluate the satisfiability of a future-time MLTL formula φ at time stamp i in a trace π , the RV engine needs to know the evaluation of atomic propositions at future time stamps. Real sensor data collected as execution progresses can be useful to determine a verdict (true or false). However, it might require significant data collection and time that can prolong the mitigation system action (in case the formula is not satisfiable). MPRV uses a model of the runtime system to complement real-data collection with predicted values of atomic propositions at future time stamps.

Definition 5. (Model Predictions) Let φ be an MLTL formula over atomic propositions $\mathcal{AP}$ and trace π . Let $p(i)$ denote assignment to $p \in \mathcal{AP}$ at time stamp i in trace π . Given a system model M , predictive controller C , and $p(i), p(i-1), p(i-2) \dots$, model prediction estimates assignments $p(j)$ for $j > i$.

Fig. 4 shows a high-level overview of MPRV. The future-time monitor utilizes real sensor data and model predictions to evaluate the satisfiability of formula φ . The quality of the predictions (fewer false positives) depends on the type of controller and modeling inaccuracies within the model predictor. The goal of MPRV is to determine the most-likely satisfiability result of φ at time stamp i (as soon as possible) such that the supervisory controller can take the appropriate mitigation action. MPRV uses a mitigation trigger time deadline $d \in \mathbb{Z}$ by which we must determine the two-valued verdict of an MLTL formula φ . The deadline d is measured relative to a position i in the finite trace π ; we need to determine whether $\pi, i \models \varphi$, no later than d time steps after i .

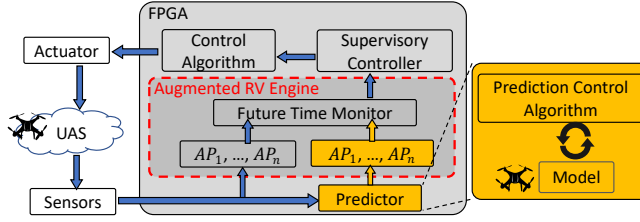


Fig. 4. High-level overview of Model Predictive Runtime Verification (MPRV). The blue and yellow arrows represent real and predicted data values.

Illustrative Example. Consider the UAS example from Section 1. A parachute is deployed when the UAS's motors fail (mitigative action) to ensure safe landing. If the MLTL formula $\varphi = \Box_{[0,6]}a \wedge \Diamond_{[0,8]}b$ evaluates to **false**, this indicates motor failure. Fig. 5 shows the runtime trace π . Note that for $i = 0$, $\pi[0, 8]$ is the maximum trace segment required to determine satisfiability. Let d be the deadline to trigger the deployment of the parachute. MPRV monitors atoms a and b based on real and predicted data. If φ evaluates to **false** at $t_R \leq d$ based on real data, then the parachute is deployed. However, there may not be enough real data at time d (e.g., because we do not have data for the future), and the runtime monitor requires additional predicted data to make an on-deadline evaluation. MPRV incrementally queries the model predictor to populate $\pi(t_R + 1), \pi(t_R + 2), \dots, \pi(8)$ with predicted values of a and b (the number of time stamps for which predictions can be made is bounded and depends on model predictor's controller). Parachute deployment is triggered if φ evaluates to **false** as π is incrementally populated by the predictor. Since $t_R \leq d$, MPRV ensures that the mitigation action is triggered before the deadline and the UAS lands safely.

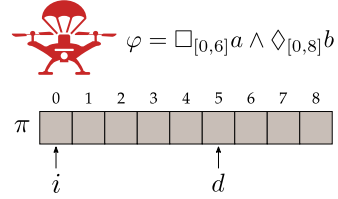


Fig. 5. Illustrative example

The goal of MPRV is to evaluate whether $\pi, i, d \models \varphi$ (the satisfiability of MLTL formula φ on computation π starting from index $i \geq 0$ can be determined within deadline d). MPRV estimates the minimum set of predicted variable valuations needed to return a verdict. The final verdict can be delayed due to temporal operators in MLTL.

Definition 6. (*Propagation Delay*) Given an MLTL formula φ and computation π , let $\hat{v}$ be the exact time stamp when execution sequence tuple T_φ is produced to determine whether $\pi, i \models \varphi$. The propagation delay of formula φ , denoted $\varphi.pd$ is the number of time stamps between i (including i) and $\hat{v}$ i.e., $\varphi.pd = \hat{v} - i$.

Definition 7. (*Best-case Propagation Delay*) The best-case propagation delay of an MLTL formula φ , denoted $\varphi.bpd$, is the minimum propagation delay required to determine whether $\pi, i \models \varphi$ at time stamp $\hat{v}$, i.e., $\varphi.bpd = \min_{\hat{v}}(\varphi.pd)$.

Definition 8. (*Worst-case Propagation Delay*) The worst-case propagation delay of an MLTL formula φ , denoted $\varphi.wpd$, is the maximum propagation delay required to determine whether $\pi, i \models \varphi$ at time stamp $\hat{v}$, i.e., $\varphi.wpd = \max_{\hat{v}}(\varphi.pd)$.

The minimum trace segment required for evaluation in the best case is $\pi[i, i + \varphi.bpd]$ if $\varphi.bpd \geq 0$ or $\pi[i + \varphi.bpd, i]$ if $\varphi.bpd < 0$; in the worst case it is $\pi[i, i + \varphi.wpd]$.

Therefore, the trace segment required to conclude whether $\pi, i \models \varphi$ is contained between $\pi[i + \varphi.bpd, i + \varphi.wpd]$. In our illustrative example, the worst-case propagation delay is $\varphi.wpd = 8$, while the best-case propagation delay is $\varphi.bpd = 0$ with a model predictor that can predict eight time stamps in the future.

Definition 9. (Partial Evaluation) *The partial evaluation of an MLTL formula φ over trace π at time stamp $i \geq 0$, denoted as $\varphi|_{(\pi, i, d)}$, is the evaluation of $\pi, i \models \varphi$ based on the trace segment $\pi[i, i + d]$.*

The symbol ' $|$ ' has a similar meaning to that in propositional satisfiability. There are two possible outcomes to partial evaluation: 1) $d \geq \varphi.wpd$: in this case $\varphi|_{(\pi, i, d)}$ is either true or false and enough to evaluate $\pi, i, d \models \varphi$, and 2) $d < \varphi.wpd$: in this case we simplify φ with variables assignments in $\pi[i, i + d]$ and standard logic rewriting rules. Going back to our illustrative example, we have $\varphi = (\Box_{[0,6]}a) \wedge (\Diamond_{[0,8]}b)$, $i = 0$, $d = 5$, and let evaluations of a and b be $\pi_{\langle a, b \rangle} = [\langle 1, 0 \rangle, \langle 1, 1 \rangle, \langle 1, 0 \rangle, \langle 1, 1 \rangle, \langle 1, 0 \rangle, \langle 1, 0 \rangle]$. Since the subformula $\Diamond_{[0,8]}b$ is satisfied at time stamp 1, we have $\varphi|_{(\pi, 0, 5)} = \Box_{[5,6]}a$ and we predict value for a at time stamp 6 to evaluate $\pi, i, d \models \varphi$.

Definition 10. (Prediction Horizon) *Given a predictive controller C , the prediction horizon, denoted H_p , is the number of time stamps in trace π , starting from the current time stamp t_R , that are populated using predictions from C to generate $\pi[t_R, t_R + H_p]$. The maximum prediction horizon is denoted by $\max(H_p)$.*

Note that maximum prediction horizon $\max(H_p)$ is a property of the controller used for prediction. Any predictive controller can be used based on the chosen risk profile.

Definition 11. (Model Predictive Runtime Verification (MPRV)) *Given an MLTL formula φ and a deadline d , MPRV produces an execution sequence $\langle T_{\varphi @ d} \rangle$ of two-valued tuples $T_{\varphi @ d} = (v, \tau)$ for evaluating φ over trace π at index i such that tuple $T_{\varphi @ d}$ with $T_{\varphi @ d}.\tau \geq i$ is produced no later than deadline $i + d$. It populates a predicted trace $\hat{\pi}$, by increasing H_p , using a future state estimation technique as follows:*

- $d \geq \varphi.wpd$: $T_{\varphi @ d} \equiv ((\pi, i, d \models \varphi), i)$ (no prediction is required)
- $d < \varphi.bpd$: $T_{\varphi @ d} \equiv ((\hat{\pi}, i, d \models \varphi|_{(\pi, i, d)}), i)$ (relies on prediction $\hat{\pi}$)
- otherwise: $T_{\varphi @ d} \equiv ((\hat{\pi}, i, d \models \varphi|_{(\pi, i, d)}), i)$ (may need prediction $\hat{\pi}$)

We assume estimation takes into account all variable assignments from π based on real data. We introduce deadline d in the execution sequence because different components may have separate deadline requirements. The deadline in the execution sequence is required to compute the necessary prediction horizon for π . Lemma 1 and 2 guarantee correctness of MPRV and establish bounds on the produced execution tuples.

Lemma 1. *Given an MLTL formula φ , a trace π , and a deadline d , MPRV can produce execution sequence tuples $T_{\varphi @ d}$ for all time stamps $t_R \geq \varphi.wpd$ and trace segments $\pi[0, t_R]$ such that $0 \leq T_{\varphi @ d}.\tau \leq t_R - \varphi.wpd$.*

Lemma 2. *Given an MLTL formula φ , a trace π , and a deadline d , MPRV can produce execution sequence tuples $T_{\varphi @ d}$ for all time stamps t_R and trace segments $\pi[0, t_R]$ such that if $T_{\varphi @ d}.v$ is produced at time stamp t_R then $T_{\varphi @ d}.\tau \in [t_R - \varphi.wpd, t_R - \varphi.bpd]$.*

3.1 Algorithm and Implementation

We contribute an algorithm MPRV along with an implementation in the state-of-the-art RV engine R2U2. In the compile stage, we topologically sort the nodes in the AST: φ_{AST} by setting a higher priority to the child node. This offline step generates the custom instructions as in Fig. 2. We use the sorted node list: $TpO(\varphi_{AST})$ to represent the sequence of the execution. Algorithm 1 shows the procedure for MPRV at each time stamp. During runtime, the system takes the digital sensor signals as the input. These sensor signals are converted into the Boolean variables for trace π as is shown in Algorithm 3. Then we start the normal RV in R2U2. Algorithm 2 is the brief version of the MLTL operator algorithm from [37]. The verification results are stored in the queue. Since those queues are also used for storing the predicted verification execution sequence, the data content that can be potentially modified in the queue is stored into another memory space. As a feature of our design, the data entry that we need to store is equal to the number of the custom instructions, which is independent of the size of the original R2U2 queue. After the caching, the prediction part starts. The predictive RV terminates until the partial evaluation of φ evaluates to true or false regarding time stamp i . Finally, we restore the previously stored data content. All of these tasks should be completed in one sensor sampling period according to the asynchronous observer requirement. We prove that the algorithm terminates (line 7~12 in Algorithm 1) by proving an upper bound on the maximum prediction horizon $\max(H_p)$. This is the same as deciding the value of H_p that satisfies the MPRV requirement: $T_{\varphi@d}$ with $T_{\varphi@d}.\tau \geq i$, must be produced by the absolute deadline $i + d$. The output of the MPRV algorithm is the execution sequence combined by the result from the real signal and the result from the predicted signal. A false verdict in the result indicates a violation to the specification.

Algorithm 1: procedure for MPRV (Def. 11) at each time stamp i . MLTL operators' local variables are defined in the implementation's algorithms[22]

```

Init:  $H_p = 0$ ; // prediction horizon set to 0 (no prediction)
input : Signal:  $s_{real}$ ; Trace:  $\pi[0, i - 1]$  or empty  $\pi$ ; Deadline:  $d$ ; AST:  $\varphi_{AST}$ ;
1 update_trace( $s_{real}, \pi, i, 0$ ); // run signal through the atomic checkers to populate the
   atomic propositions
   // select  $g$  from the sorted node list
2 for Node  $g$  in  $TpO(\varphi_{AST})$  do
3    $node\_operate(\pi, i, g)$ ; // run operator algorithm on the Node
   // record information for executing next time stamp  $i + 1$ 
4   Record Node  $g$ 's Queue data;
5   Record Node  $g$ 's operator's local variables;
6  $\varphi|_{\pi, i, d} \leftarrow read(\varphi.Queue)$ ; // the output from the top Node  $\varphi$  of  $\varphi_{AST}$  is the RV result of
   the MLTL formula
7  $\hat{\pi} \leftarrow \pi$ ; // initialize  $\hat{\pi}$  with the real data (no prediction)
   // check if we need prediction; if not we are finished
8 while  $\varphi|_{\hat{\pi}, i, d} \notin \{true, false\}$  do
   // loop until we can assert true or false at time stamp  $i$ 
9    $H_p \leftarrow H_p + 1$ ; // look into the next prediction step
10  update_trace( $s_{real}, \hat{\pi}, i, H_p$ ); // add predicted data to trace
11  for Node  $g$  in  $TpO(\varphi_{AST})$  do
12     $node\_operate(\hat{\pi}, i + H_p, g)$ 
13   $\varphi|_{\hat{\pi}, i, d} \leftarrow read(\varphi.Queue)$  // get the RV result at prediction time stamp  $i + H_p$ 
   // restore information for executing next time stamp  $i + 1$ 
14 for Node  $g$  in  $TpO(\varphi_{AST})$  do
15   Restore Node  $g$ 's Queue data;
16   Restore Node  $g$ 's operator's local variables;
17 return  $\varphi|_{\hat{\pi}, i, d}$ ;

```

show the data content of $g.Queue$ and all other queues from g 's sibling nodes. We look for the maximum possible value of k to decide the memory usage of $g.Queue$. First, we analyze nodes g and g_1 as child nodes of h_1 . Execution sequence tuples with the same τ are consumed/popped from $g.Queue$ and $g_1.Queue$ to produce the output tuples. These tuples (marked in dashed boxes) can be dropped from the perspective of being h_1 's input data before the system knows $\pi[t_R]$. Note that $\pi[0, t_R - 1]$ is known and $\pi[t_R]$ is unknown for the previous time stamp. According to Lemma 1, if $\pi[0, t_R - 1]$ is known but $\pi[t_R]$ is unknown, T_{g_1} can be produced with $T_{g_1}.\tau \geq t_R - 1 - g_1.wpd$. Therefore $y_1.\tau \geq t_R - 1 - g_1.wpd$. Similarly, according to Lemma 2, when $\pi[0, t_R]$ is known, if there are new execution sequence tuples T_g coming into $g.Queue$, then we have $T_g.\tau \in [t_R - g.wpd, t_R - g.bpd]$. Therefore, $x.\tau \leq t_R - g.bpd$. The maximum value of k is $\max(x.\tau - y_1.\tau)$, which is $\max(x.\tau) - \min(y_1.\tau) = t_R - g.bpd - (t_R - 1 - g_1.wpd) = g_1.wpd - g.bpd + 1$. The maximum number of elements to be stored in $g.Queue$ is bounded by $g_1.wpd - g.bpd + 1$, or 1 if the difference is non-positive. We consider not only g_1 but also $g_2, \dots, g_p$ and apply the same analysis. Therefore, we have $g.Queue$ bounded by $s.wpd - g.bpd + 1$ for $s \in \mathbb{S}_g = \{g_1, \dots, g_p\}$. The queue size is computed as below:

$$size(g.Queue) = \max(\max\{s.wpd \mid \forall s \in \mathbb{S}_g\} - g.bpd, 0) + 1 \quad (1)$$

For deciding the memory usage in Algorithm 4, we first derive $TpO(\varphi_{AST})$. Second, we compute the bpd and wpd of each operator from the bottom to the top node on the AST. Finally, we compute the output queue size for each subformula using (1).

Algorithm 4: Determine the size of queues

Input: MLTL: φ , Prediction Length: H_p

```

1 for Node  $g$  in  $TpO(\varphi_{AST})$  do
  // bottom-up loop through all the nodes
2   Compute  $g.bpd$  and  $g.wpd$ ;                                     // based on Definition 7,8
3 for Node  $g$  in  $\varphi_{AST}$  do
  // assign size to all the nodes
4   Compute  $size(g.Queue)$ ;                                       // based on Formula 1
```

Assume $\max(\tau)$ is the largest time stamp the clock can reach. Each queue entry stores a tuple of a verdict and a time stamp, where we use one bit represents the verdict and $\lceil \log_2 \max(\tau + 1) \rceil$ bits represent the time stamp. The total memory size (in bits) is:

$$(1 + \lceil \log_2 \max(\tau + 1) \rceil) \times \sum_{g \in \varphi_{AST}} size(g.Queue)$$

Worst Case Execution Time (WCET) The MPRV execution time is split into two parts: the model prediction and predictive RV. In this paper, we only analyze the WCET of RV (including RV on the predicted trace). The computation time for predicting the future states is not taken into account for two reasons: 1) the model prediction time depends on the exact control algorithm and size of the system model, which is not a focus in this paper; 2) the model prediction can potentially run in parallel with the predictive RV. The WCET for verification is a function regarding the property specification formula φ and the prediction step H_p . We recall that in Section 2.2, φ is split into a set of subformulas in the form of nodes in an AST. The WCET is the sum of execution time of all the nodes. So here we first analyze the execution time of a single node. For a unary (binary) operator, say node g , the forward pointer(s) is/are used in the algorithm to scan through the input queue(s). Thus the execution time of node g is bounded by the sum of its input queue(s). We use $size(g.iQueue)$ to denote the total size of the input

queue(s) connected to node g . We let t_{exe} be the time for executing the operator logic. Since we are running H_p loops, we multiply the time by H_p . The worst case execution time of our proposed MPRV at each time stamp for formula φ is given by the following formula:

$$WCET(\varphi) = \left\{ \sum_{\mathcal{N} \in \varphi_{AST}} size(\mathcal{N}.iQueue) \times t_{exe} \right\} \times H_p + t_{store} + t_{restore} \quad (2)$$

In our proposed architecture, the execution time t_{exe} is bounded by $0.39\mu s$ ¹. We use t_{store} as the time for storing the running status and $t_{restore}$ as the time for restoring the status. A feature of the implementation is that the system does not need to store the data content in the queue. Since the queue size has been increased by taking account the prediction step H_p , it guarantees the tuples generated during prediction will not overwrite the valid non-predicted data content in the queue. Thus t_{store} and $t_{restore}$ are fixed for each observer and are negligible compared with t_{exe} from our observation.

4 Evaluation

We illustrate MPRV for a UAS quadcopter tracking a given trajectory. A discrete state-space model is used to describe and predict the future behavior of the quadcopter. We also discuss how our approach is feasible for a wide range of systems.

A generic linearized discrete state-space model consists of matrices A , B , C , and D ² and is formulated as follows:

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k + w_k \\ y_k &= Cx_k + Du_k + v_k \end{aligned} \quad (3)$$

- $x_k \in \mathbb{R}^N$: a vector represents the state of the system at time k
- $u_k \in \mathbb{R}^M$: a vector represents the input acting on the system at time k
- $y_k \in \mathbb{R}^P$: a vector represents outputs of the system at time k
- A is a $N \times N$ matrix that defines the internal dynamics of the system
- B is a $N \times M$ matrix that defines how the input acting upon the system input
- C is a $P \times N$ matrix that transforms states of the system into outputs (y_k)
- w_k and v_k are $N \times 1$ and $P \times 1$ vectors that represents zero-mean noise signals

N is the number of state variables, M is the number of inputs and P is the number of observable state variables from the system. Matrix A represents the dynamics of the system being controlled, matrix B represents how actuator commands (i.e., u_k) impact the system, and the matrix C is a mapping of the current state to the output obtained from sensors (i.e., y_k).

We simulate a quadcopter UAS whose trajectory is controlled via a Model Predictive Controller (MPC); refer [30] for information about MPC control algorithm. The quadcopter model we use in our system is obtained from [24], including the default parameters presented for the model. The states of the quadcopter (x_k), the inputs to quadcopter (u_k), and the measurable outputs of the quadcopter (y_k) are:

$$\begin{aligned} x_k^T &= [\Phi, \dot{\Phi}, \Theta, \dot{\Theta}, \Psi, \dot{\Psi}, z, \dot{z}, x, \dot{x}, y, \dot{y}]^T \\ u_k^T &= [(T_1 + T_2 + T_3 + T_4), (T_2 - T_4), (T_1 - T_3), ((H_1 + H_3) + (H_2 + H_4))]^T \\ y_k^T &= [\Phi, \Theta, \Psi, z]^T \end{aligned}$$

¹ Based on our hardware running at a clock frequency of 100 MHz.

² It is common to omit the matrix D , as inputs typically do not directly impact output.

where Φ, Θ, Ψ , represent the roll, pitch, and yaw angles of the quadcopter (with respect to a body-frame axis) and x, y, z represent the Cartesian positions of the quadcopter (with respect to an inertial axis). Additionally, T_i and H_i correspond to motor i 's thrust force and hub torque, respectively. To save space, we refrain from presenting any more details about the linear state-space model for the quadcopter and encourage the reader to turn to [24] for more details.

We use an MPC controller to guide the quadcopter along a predefined trajectory. The key point of MPC is to minimize a cost function subject to constraints. In our simulation, each element in input u_k is constrained to the range $[-10, 10]$. The absolute value of x_k vector element is constrained within $(\frac{\pi}{6}, \frac{\pi}{6}, \frac{\pi}{6}, \frac{\pi}{12}, \frac{\pi}{6}, \frac{\pi}{6}, +\infty, 1, +\infty, 1, +\infty, 1)$. To minimize the cost function, we associate a multiplicative cost weighting matrix of $diag(0, 0, 0, 0, 0, 0, 150, 0, 8, 8, 8, 8)$ for x_k , and $diag(10, 10, 10, 10)$ for u_k , where $diag$ denotes a square matrix with the elements listed on the diagonal. The MPC prediction horizon is set to 20 update steps into the future, and the controller actuation update rate to 50 Hz. The resulting trajectory of the quadcopter compared to the planned trajectory is shown in the upper portion of Fig. 7.

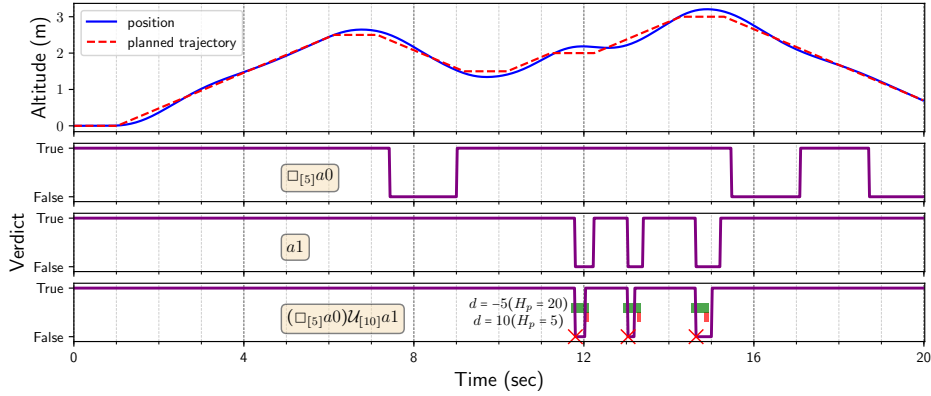


Fig. 7. Model predictive control of the position of a quadcopter, and verification results of three formulas. The colored bars in the bottom figure indicate the time when the system detects the error marked by red $\times$. The right end of each bar is the formula-violation detection-time using RV without prediction. The left end of each green (orange) bar is the formula-violation detection-time using MPRV with a prediction horizon of H_p .

Let trace π be a combination of predicates $a0$ and $a1$ at each time stamp. Table 2 specifies the following Boolean mapping function $\mathcal{M}$ to map system states to atomics.

Atomic	Definition
$a0$	Vertical speed ($\dot{z}_k$) ≥ -0.5 m/s
$a1$	Magnitude of trajectory error ($z_{ref} - z$) ≤ 0.17 m

Table 2. Quadcopter Boolean Mapping

These atomics are used within three MLTL formulas that are specified and evaluated in Fig. 7. For formula φ in each figure, if the verdict is **true**, it means that $\pi, t \models \varphi$, where t is the corresponding x-axis value. The verdict combines the execution sequence output from the algorithm with time. Due to the nature of future time asynchronous

observers, the evaluation of $\pi, t \models \varphi$ is not known until a time later than t . Fig. 7 shows the “real-time” at which we can estimate a formula becoming **false** using colored bars. Fig. 7 also compares the responsiveness of formula $\varphi = (\Box_{[5]}a0)\mathcal{U}_{[10]}a1$ using deadline $d = -5$ steps (0.1s before violation) and $d = 10$ steps (0.2s after violation). Since the $\varphi.wpd = 15$, we compute the required prediction steps using Lemma 3. The results are $H_p = 20$ when $d = -5$ and $H_p = 5$ when $d = 10$. The green bar is associated with a 20 step prediction horizon. The left boundary of the green bar is the time when MPRV detects the formula becoming **false**, and right boundary is the time when a **false** valuation is detected without prediction. Note that for $d = -5$, the green bars maintain at least 0.1s ahead of the corresponding **false** verdict (marked by red cross). Similarly for a $H_p = 5$, the orange bar depicts similar behavior.

In summary, the longer the prediction horizon, the earlier conclusive results are obtained. These results are in alignment with what one would intuitively expect. That is, as one estimates further into the future, the earlier one can make predictions about future behavior. Given perfect sensors, and perfect knowledge about the operating environment, these are reasonable results. Of course, real-world sensors and imperfect knowledge limit how far one can effectively estimate the future.

Models for Prediction Our methodology is applicable across a wide range of systems. When targeting a given system, a user would want to deploy their desired MLTL formulas and system models. Deployment of formulas, in our implementation of MPRV, uses R2U2’s procedure for specifying MLTL formulas. Many examples exist in literature of ways to develop models for control and prediction. The most common method for developing models is to derive a differential-algebraic model by analyzing the physical system’s dynamics [11,9]. Another common modeling method is system identification: deriving the model of the system by observing the system’s inputs and corresponding outputs [18,34]. Data-driven modeling, i.e., machine learning [45] or simulation-based modeling [21,7], are a few more popular modeling techniques. We provide details on how to deploy a new system model, and to determine if MPRV is feasible for a given embedded platform’s constraints and mission requirements below.

4.1 Implementation of Model Predictive Runtime Verification (MPRV)

For experimental evaluation of our methodology, we target a modest sized Xilinx FPGA (ZYNQ 7020) [1] and use Vivado 2019.2 tool-chain to synthesize our MPRV hardware design: both an augmented RV engine (R2U2 with MPRV) and an MPC controller for signal prediction. The resource usage is shown in Fig. 8. While we target a running frequency of 100MHz, Vivado’s synthesis reports the design is able to run at 130MHz. Note that our implementation of the predictive RV engine and the MPC are modular in their design: several software configurable registers allow the user to quickly modify the predictive RV engine’s formulas as well as the MPC’s control algorithm on-the-fly without having to re-synthesize the hardware. Additionally, the MPC hardware design has a trade-off between performance and resource utilization; we have maximized the MPC design to fill up the remaining resources the predictive RV engine did not use. One example of this performance/resource trade-off is in the MPC’s matrix-vector multiplier (MVM). The larger the MVM, the faster the MPC can handle larger matrices, i.e., the longer the MPC’s prediction horizon or the larger the system size N . An MLTL

formula is represented as an abstract syntax tree (AST), where sub-formulas are depicted as nodes. To verify an MLTL formula at run-time, we encode each node as an asynchronous observer. We employ queuing structures that we refer to as Shared Connection Queues (SCQs) to buffer and communicate results produced by child nodes for consumption by their parents. Unlike a traditional First In, First Out (FIFO) structure, SCQ is a circular buffer consisting of one write pointer and one or more read pointers. For our use-case, given an MLTL formula, sizing SCQs in accordance to the procedure given in Section 3.2 guarantees SCQs never overflow. In our predictive RV engine, we can specify the time-stamp width and total SCQ size. For our quadcopter example, we chose a time stamp width of 16-bits and a total SCQ size of 2048 bits, which is represented as a single block RAM (BRAM). Figure 8 shows our augmented RV engine also uses additional BRAMs for instruction and local variable RAM (local variables are defined in [22]), as well as data transmission FIFOs.

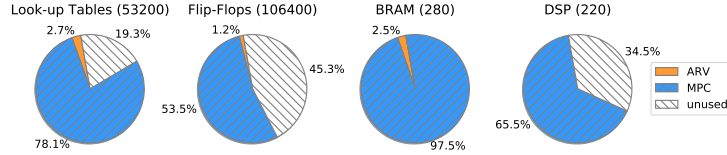


Fig. 8. Percentage of resource usage by augmented RV engine, MPC, and unused resource. The total amount of each resource available on chip is included inside of the braces. For the BRAM, the blue corresponds to a system of 12 state variables (N) with $H_p = 91$; one can trade memory usage between N and H_p (see Figure 9).

For MPRV (assuming an MPC controller), we can target a given system by providing appropriate state space matrices (i.e., A , B , C of Formula 3). For example, when embedding MPRV into a quadcopter, we could use the model from [23] derived from the vehicle’s physical dynamics. Hardware architectures exist that support on-line configuration of MPC for different systems, e.g., [48]. Assuming the embedded hardware-based MPC controller of [48] acting as the prediction engine of our MPRV architecture, we show in Fig. 9 our approach scales feasibly across a wide range of systems. To give a sense of the range of system complexity our approach can support, we highlight a few examples in terms of number of system model state variables (N). If we limit the upper bound of N to 30, we can perform MPRV for a quadcopter [24] ($N = 12$), a car with active suspension ($N = 18$), and even a reduced order model of a Fusion Machine [8] ($N = 2700$ for full model, but $N = 30$ for reduced model at an accuracy loss of .1%).

The left-hand side of Fig. 9 depicts how memory usage scales with N , and with the length of the prediction horizon (H_p). The right-hand side of Fig. 9 depicts how computation-time scales with N and H_p . Additionally, the right-hand side of Fig. 9 illustrates how one would take into account a system’s required sensor update rate (driven physical system dynamics) to determine how far into the future we can predict (H_p). This is accomplished using the two horizontal dotted lines located at .01 seconds (10ms), and at .05 seconds (50ms). For example, if a system requires a sensor update rate of .05 seconds, then for a quadcopter system of size $N = 12$ one could look a horizon of $H_p = 10$ or $H_p = 50$ into the future, but not $H_p = 100$ as computation requires more than .05 seconds. If this same sized system required a sensor update rate of .01 seconds, a prediction horizon of $H_p = 10$ would be computed fast enough to

meet system dynamic constraints, but a horizon of $H_p = 50$ would not be. In summary, Fig. 9 can be used to help designers/users determine early in a design process and/or at run-time whether an embedded platform’s memory, and computing capabilities are sufficient to support MPRV, given their system requirements, and can help with reasoning about system performance/resource trade-offs when using MPRV.

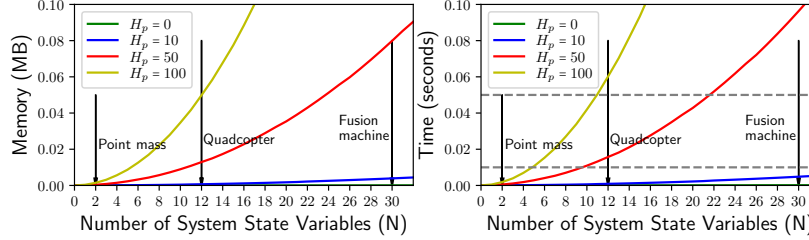


Fig. 9. Memory and computation-time of MPC from [48]. The point mass has $N = 2$ state variables. The quadcopter model [24] has $N = 12$. The fusion machine [8] has $N = 30$.

5 Conclusion

The definition and general algorithm for MPRV are *extensible*; while we have chosen to exemplify MPRV using MLTL properties verified in the R2U2 engine with a model-predictive controller for a UAS, all of these choices can be changed or extended. Our results are promising in terms of MPRV’s utility for improving RV responsiveness in real-life scenarios. In particular, our implementation paves the way for better mitigation of faults by enabling future-time requirements to be evaluated with enough time to trigger mitigation actions during system runtime. There is now a basis to extend MPRV to other logics (besides MLTL), create implementations that build on other RV engines (with the caveat that this may extend R2U2’s minimal resource consumption), and plug in different models and controllers for when prediction steps are necessary. Investigating the trade-offs between resource demands, performance, and accuracy of different models and controllers in the MPRV setting could map a valuable landscape for UAS design decisions going forward.

MPRV extensibility is essentially bounded by the following set of limitations. The system needs to guarantee that run mode change can take effect within the prediction horizon. We can toggle prediction horizon with the caveat that there are some notable drawbacks to long prediction steps: 1) the computation should be completed within one controller sample period; 2) the prediction is prone to make errors in the future; 3) errors due to the data processing precision will be augmented under loops of a prediction.

MPRV only scratches the surface of the many aspects at the intersection of control theory and RV left to explore. For example, in terms of minimizing platform resources, there exists a co-design trade-off opportunity between RV specification and controller constraint complexity. Avenues to optimize performance include exploring an application-specific architecture, and exploring explicit model-predictive control (MPC) [3] to speed up the future state prediction. We look forward to exploring ways to mitigate inaccuracies in the model and/or controller used for prediction, such as using reinforcement learning against the pair of predicted and actual traces to iteratively refine our controllers through RV, thereby increasing their accuracy over time.

References

1. Zynq-7000 soc data sheet: Overview. https://www.xilinx.com/support/documentation/data_sheets/ds190-Zynq-7000-Overview.pdf (2018)
2. Adolf, F.M., Faymonville, P., Finkbeiner, B., Schirmer, S., Torens, C.: Stream runtime monitoring on uas. In: International Conference on Runtime Verification. pp. 33–49. Springer (2017)
3. Alessio, A., Bemporad, A.: A survey on explicit model predictive control. In: Nonlinear model predictive control, pp. 345–369. Springer (2009)
4. Alur, R., Henzinger, T.A.: Real-time Logics: Complexity and Expressiveness. In: LICS. pp. 390–401. IEEE (1990)
5. Babae, R., Gurfinkel, A., Fischmeister, S.: $\mathcal{P}$ revent: A predictive run-time verification framework using statistical learning. In: International Conference on Software Engineering and Formal Methods. pp. 205–220. Springer (2018)
6. Babae, R., Gurfinkel, A., Fischmeister, S.: Predictive run-time verification of discrete-time reachability properties in black-box systems using trace-level abstraction and statistical learning. In: International Conference on Runtime Verification. pp. 187–204. Springer (2018)
7. Banaei, M.R., Alizadeh, R.: Simulation-based modeling and power management of all-electric ships based on renewable energy generation using model predictive control strategy. IEEE Intelligent Transportation Systems Magazine **8**(2), 90–103 (2016)
8. Bonotto, M., Bettini, P., Cenedese, A.: Model-order reduction of large-scale state-space models in fusion machines via krylov methods. IEEE Transactions on Magnetics **53**(6), 1–4 (2017)
9. Brown, R.G., Hwang, P.Y.C.: Introduction to Random Signals and Applied Kalman Filtering with MATLAB Exercises. John Wiley & Sons, Inc., 4th edn. (2012), ISBN-13 978-0-470-60969-9
10. Cassar, I., Francalanza, A., Aceto, L., Ingólfssdóttir, A.: A survey of runtime monitoring instrumentation techniques. arXiv preprint arXiv:1708.07229 (2017)
11. Chen, C.: Oxford University Press, Inc., 3rd edn. (1999), ISBN-13 978-0-19-511777-6
12. D’Angelo, B., Sankaranarayanan, S., Sanchez, C., Robinson, W., Finkbeiner, B., Sipma, H.B., Mehrotra, S., Manna, Z.: Lola: runtime monitoring of synchronous systems. In: 12th International Symposium on Temporal Representation and Reasoning (TIME’05). pp. 166–174 (2005)
13. Dokhanchi, A., Hoxha, B., Fainekos, G.: On-line monitoring for temporal logic robustness. In: International Conference on Runtime Verification. pp. 231–246. Springer (2014)
14. Erzberger, H., Heere, K.: Algorithm and operational concept for resolving short-range conflicts. Proc. IMechE G J. Aerosp. Eng. **224**(2), 225–243 (2010). <https://doi.org/10.1243/09544100JAERO546>, <http://pig.sagepub.com/content/224/2/225.abstract>
15. Etigowni, S., Hossain-McKenzie, S., Kazerooni, M., Davis, K., Zonouz, S.: Crystal (ball): I look at physics and predict control flow! just-ahead-of-time controller recovery. In: Proceedings of the 34th Annual Computer Security Applications Conference. pp. 553–565. ACM (2018)
16. Falcone, Y., Krstić, S., Reger, G., Traytel, D.: A taxonomy for classifying runtime verification tools. In: International Conference on Runtime Verification. pp. 241–262. Springer (2018)
17. Finkbeiner, B., Kuhtz, L.: Monitor circuits for ltl with bounded and unbounded future. In: International Workshop on Runtime Verification. pp. 60–75. Springer (2009)
18. Greblicki, W.: Continuous-time hammerstein system identification from sampled data. IEEE Transactions on Automatic Control **51**(7), 1195–1200 (2006)

19. Heffernan, D., Macnamee, C., Fogarty, P.: Runtime verification monitoring for automotive embedded systems using the iso 26262 functional safety standard as a guide for the definition of the monitored properties. *IET Software* **8**(5), 193–203 (October 2014). <https://doi.org/10.1049/iet-sen.2013.0236>
20. Jaksic, S., Bartocci, E., Grosu, R., Kloibhofer, R., Nguyen, T., Nickovic, D.: From signal temporal logic to FPGA monitors. In: *Formal Methods and Models for Codesign (MEMOCODE)*, 2015 ACM/IEEE International Conference on. pp. 218–227 (Sept 2015). <https://doi.org/10.1109/MEMCOD.2015.7340489>
21. Kapinski, J., Deshmukh, J.V., Jin, X., Ito, H., Butts, K.: Simulation-based approaches for verification of embedded control systems: An overview of traditional and advanced modeling, testing, and verification techniques. *IEEE Control Systems Magazine* **36**(6), 45–64 (2016)
22. Kempa, B., Zhang, P., Jones, P.H., Zambreno, J., Rozier, K.Y.: Embedding online runtime verification for fault disambiguation on robonaut2. In: *International Conference on Formal Modeling and Analysis of Timed Systems*. pp. 196–214. Springer (2020)
23. Kong, J., Pfeiffer, M., Schildbach, G., Borrelli, F.: Kinematic and dynamic vehicle models for autonomous driving control design. In: *2015 IEEE Intelligent Vehicles Symposium (IV)*. pp. 1094–1099. IEEE (2015)
24. Kurak, S., Hodzic, M.: Control and estimation of a quadcopter dynamical model. *Periodicals of Engineering and Natural Sciences* **6**(1), 63–75 (2018)
25. Lee, P.P., Bu, T., Chandranmenon, G.: A lock-free, cache-efficient shared ring buffer for multi-core architectures. In: *Proceedings of the 5th ACM/IEEE Symposium on Architectures for Networking and Communications Systems*. pp. 78–79. ACM (2009)
26. Leucker, M.: Sliding between model checking and runtime verification. In: *International Conference on Runtime Verification*. pp. 82–87. Springer (2012)
27. Leucker, M., Sánchez, C., Scheffel, T., Schmitz, M., Thoma, D.: Runtime verification for timed event streams with partial information. In: *International Conference on Runtime Verification*. pp. 273–291. Springer (2019)
28. Li, J., Vardi, M.Y., Rozier, K.Y.: Satisfiability checking for mission-time LTL. In: *International Conference on Computer Aided Verification*. pp. 3–22. Springer (2019)
29. Lu, H., Forin, A.: The Design and Implementation of P2V, An Architecture for Zero-Overhead Online Verification of Software Programs. Tech. Rep. MSR-TR-2007-99, Microsoft Research (August 2007), <http://research.microsoft.com/apps/pubs/default.aspx?id=70470>
30. Maciejowski, J.M.: *Predictive control: with constraints*. Pearson education (2002)
31. Maler, O., Nickovic, D., Pnueli, A.: On synthesizing controllers from bounded-response properties. In: *International Conference on Computer Aided Verification*. pp. 95–107. Springer (2007)
32. Maler, O., Nickovic, D., Pnueli, A.: Checking Temporal Properties of Discrete, Timed and Continuous Behaviors, pp. 475–505. Springer Berlin Heidelberg, Berlin, Heidelberg (2008), http://dx.doi.org/10.1007/978-3-540-78127-1_26
33. Meredith, P.O., Jin, D., Griffith, D., Chen, F., Roşu, G.: An overview of the MOP runtime verification framework. *International Journal on Software Tools for Technology Transfer* **14**(3), 249–289 (2012)
34. Naung, Y., Schagin, A., Oo, H.L., Ye, K.Z., Khaing, Z.M.: Implementation of data driven control system of dc motor by using system identification process. In: *2018 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIConRus)*. pp. 1801–1804 (2018)
35. Pellizzoni, R., Meredith, P., Caccamo, M., Rosu, G.: Hardware runtime monitoring for dependable COTS-based real-time embedded systems. *RTSS* pp. 481–491 (2008)

36. Pinisetty, S., J  ron, T., Tripakis, S., Falcone, Y., Marchand, H., Preoteasa, V.: Predictive runtime verification of timed properties. *Journal of Systems and Software* **132**, 353–365 (2017)
37. Reinbacher, T., Rozier, K.Y., Schumann, J.: Temporal-logic based runtime observer pairs for system health management of real-time systems. In: *Proceedings of the 20th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Lecture Notes in Computer Science (LNCS), vol. 8413, pp. 357–372. Springer-Verlag (April 2014)
38. Ro  u, G., Havelund, K.: Rewriting-based techniques for runtime verification. *Automated Software Engineering* **12**(2), 151–197 (2005)
39. Ryan, J., Cummings, M., Roy, N., Banerjee, A., Schulte, A.: Designing an interactive local and global decision support system for aircraft carrier deck scheduling. In: *Infotech@ Aerospace 2011*, p. 1516
40. Schumann, J., Moosbrugger, P., Rozier, K.Y.: R2U2: Monitoring and Diagnosis of Security Threats for Unmanned Aerial Systems. In: *Proceedings of the 15th International Conference on Runtime Verification (RV15)*. Springer-Verlag, Vienna, Austria (September 2015)
41. Selyunin, K., Nguyen, T., Bartocci, E., Nickovic, D., Grosu, R.: Monitoring of MTL specifications with IBM’s spiking-neuron model. In: *2016 Design, Automation Test in Europe Conference Exhibition (DATE)*, pp. 924–929 (March 2016)
42. Selyunin, K., Nguyen, T., Bartocci, E., Grosu, R.: Applying Runtime Monitoring for Automotive Electronic Development, pp. 462–469. Springer International Publishing, Cham (2016)
43. Tiger, M., Heintz, F.: Stream reasoning using temporal logic and predictive probabilistic state models. In: *2016 23rd International Symposium on Temporal Representation and Reasoning (TIME)*, pp. 196–205. IEEE (2016)
44. Todman, T., Stilkerich, S., Luk, W.: In-circuit temporal monitors for runtime verification of reconfigurable designs. In: *Proceedings of the 52nd Annual Design Automation Conference*, pp. 50:1–50:6. DAC ’15, ACM, New York, NY, USA (2015). <https://doi.org/10.1145/2744769.2744856>, <http://doi.acm.org/10.1145/2744769.2744856>
45. Torabi, A.J., Er, M.J., Li, X., Lim, B.S., Zhai, L., Oentaryo, R.J., Peen, G.O., Zurada, J.M.: A survey on artificial intelligence-based modeling techniques for high speed milling processes. *IEEE Systems Journal* **9**(3), 1069–1080 (2015)
46. Wong, L., Arora, N.S., Gao, L., Hoang, T., Wu, J.: Oracle streams: A high performance implementation for near real time asynchronous replication. In: *2009 IEEE 25th International Conference on Data Engineering*, pp. 1363–1374 (March 2009)
47. Yoon, H., Chou, Y., Chen, X., Frew, E., Sankaranarayanan, S.: Predictive runtime monitoring for linear stochastic systems and applications to geofence enforcement for uavs. In: *International Conference on Runtime Verification*, pp. 349–367. Springer (2019)
48. Zhang, P., Zambreno, J., Jones, P.H.: An embedded scalable linear model predictive hardware-based controller using admm. In: *Application-specific Systems, Architectures and Processors (ASAP)*, 2017 IEEE 28th International Conference on, pp. 176–183. IEEE (2017)
49. Zhang, X., Leucker, M., Dong, W.: Runtime verification with predictive semantics. In: *NASA Formal Methods Symposium*, pp. 418–432. Springer (2012)
50. Zhao, Y., Rozier, K.Y.: Formal specification and verification of a coordination protocol for an automated air traffic control system. In: *Proceedings of the 12th International Workshop on Automated Verification of Critical Systems (AVoCS 2012)*. Electronic Communications of the EASST, vol. 53. European Association of Software Science and Technology (2012)
51. Zhao, Y., Rammig, F.: Model-based runtime verification framework. *Electronic Notes in Theoretical Computer Science* **253**(1), 179–193 (2009)

A Formal Proofs

Definition 12. (*Propagation Delay Semantics*) The best- and worst-case propagation delay for an MLTL formula φ is structurally defined as

$$\begin{aligned}
-\varphi \in \mathcal{AP} : & \begin{cases} \varphi.wpd &= 0 \\ \varphi.bpd &= 0 \end{cases} & -\varphi = \neg\psi : & \begin{cases} \varphi.wpd &= \psi.wpd \\ \varphi.bpd &= \psi.bpd \end{cases} \\
-\varphi = \Box_{[\varphi.lb, \varphi.ub]} \psi \text{ or } \varphi = \Diamond_{[\varphi.lb, \varphi.ub]} \psi : & \begin{cases} \varphi.wpd &= \psi.wpd + \varphi.ub \\ \varphi.bpd &= \psi.bpd + \varphi.lb \end{cases} \\
-\varphi = \varphi_1 \vee \varphi_2 \text{ or } \varphi = \varphi_1 \wedge \varphi_2 : & \begin{cases} \varphi.wpd &= \max(\varphi_1.wpd, \varphi_2.wpd) \\ \varphi.bpd &= \min(\varphi_1.bpd, \varphi_2.bpd) \end{cases} \\
-\varphi = \varphi_1 \mathcal{U}_{[\varphi.lb, \varphi.ub]} \varphi_2 \text{ or } \varphi = \varphi_1 \mathcal{R}_{[\varphi.lb, \varphi.ub]} \varphi_2 : & \begin{cases} \varphi.wpd &= \max(\varphi_1.wpd, \varphi_2.wpd) + \varphi.ub \\ \varphi.bpd &= \min(\varphi_1.bpd, \varphi_2.bpd) + \varphi.lb \end{cases}
\end{aligned}$$

where $\mathcal{AP}$ is the set of atomic propositions. Propagation delay is produced by temporal operators and H_p . Let $\pi[0, t_R]$ be the trace segment used for prediction, where t_R is the reference time stamp of the system. Note that the trace segment $\pi[0, t_R]$ may contain the previously predicted evaluation of atomic propositions. Lemmas 1 and 2 guarantee correctness of MPRV and establish time bounds on the produced execution tuples.

Lemma 1. Given an MLTL formula φ and trace π , execution sequence tuples T_φ are produced for all time stamps $t_R \geq \varphi.wpd$ and trace segments $\pi[0, t_R]$ such that $0 \leq T_\varphi.\tau \leq t_R - \varphi.wpd$.

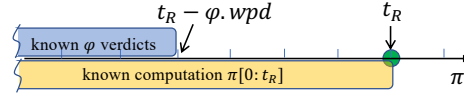


Fig. 10. Pictorial representation of Lemma 1 for *guaranteed* execution sequence elements.

Proof. We prove using induction on the structure of formula φ that T_φ with $T_\varphi.\tau \geq i$ is produced when $i = t_R - \varphi.wpd$ based on the trace $\pi[0, t_R]$ (see Fig. 10).

- $\psi \in \mathcal{AP}$: We produce T_φ with $T_\varphi.\tau \in [t_R, t_R + H_p]$ via model prediction, where H_p is the prediction horizon, based on $\pi[0, t_R]$. Since $\varphi.wpd = 0$ (definition 12), we produce T_φ with $T_\varphi.\tau \geq t_R - \varphi.wpd$.
- $\varphi = (\circ\psi)$ where $\circ \in \{\Box_I, \Diamond_I\}$: MLTL formula φ specifies a property over the interval $I = [\varphi.lb, \varphi.ub]$. Since T_ψ is known for $T_\psi.\tau \in [0, t_R - \psi.wpd]$, we produce T_φ with $T_\varphi.\tau \geq t_R - \psi.wpd - \varphi.ub$. We replace $\psi.wpd + \varphi.ub$ with $\varphi.wpd$ (definition 12), and therefore, T_φ with $T_\varphi.\tau \geq t_R - \varphi.wpd$ is produced.
- $\varphi = \neg\psi$: MLTL formula φ specifies a property over the interval $I = [\varphi.lb, \varphi.ub] = [0, 0]$. Since T_ψ is known with $T_\psi.\tau \in [0, t_R - \psi.wpd]$, we produce T_φ with $T_\varphi.\tau \geq t_R - \psi.wpd$. We replace $\psi.wpd$ with $\varphi.wpd$ (definition 12), and therefore, T_φ with $T_\varphi.\tau \geq t_R - \varphi.wpd$ is produced.
- $\varphi = (\varphi_1 \circ \psi)$ where $\circ \in \{\mathcal{U}_I, \mathcal{R}_I\}$: MLTL formula φ specifies a property over the interval $I = [\varphi.lb, \varphi.ub]$. For time stamp t that satisfies $t + \varphi.ub \leq t_R - \varphi_1.wpd$ and $t + \varphi.ub \leq t_R - \varphi_2.wpd$ we produce T_φ with $T_\varphi.\tau \geq t$, where $t \leq t_R -$

- $\max(\varphi_1.wpd, \varphi_2.wpd) - \varphi.ub$. We know $\varphi.wpd = \max(\varphi_1.wpd, \varphi_2.wpd) + \varphi.ub$ (definition 12), and therefore, T_φ with $T_\varphi.\tau \geq t_R - \varphi.wpd$ is produced.
- $\varphi = (\varphi_1 \circ \psi)$ where $\circ \in \{\wedge, \vee\}$: MLTL formula φ specifies a property over the interval $I = [\varphi.lb, \varphi.ub] = [0, 0]$. For time stamp t that satisfies $t \leq t_R - \varphi_1.wpd$ and $t \leq t_R - \varphi_2.wpd$ we produce T_φ with $T_\varphi.\tau \geq t$, where $t \leq t_R - \max(\varphi_1.wpd, \varphi_2.wpd)$. We know $\varphi.wpd = \max(\varphi_1.wpd, \varphi_2.wpd)$ (definition 12), and therefore, T_φ with $T_\varphi.\tau \geq t_R - \varphi.wpd$ is produced. $\square$

Lemma 2. Given an MLTL formula φ and trace π , execution sequence tuples T_φ are produced for all time stamps t_R and trace segments $\pi[0, t_R]$ such that if $T_\varphi.v$ is produced at time stamp t_R then $T_\varphi.\tau \in [t_R - \varphi.wpd, t_R - \varphi.bpd]$.

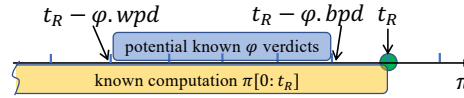


Fig. 11. Pictorial representation of Lemma 2 for range of T_φ produced at time t_R .

Proof. We prove using induction on the structure of the formula φ (see Fig. 11).

- $\psi \in \mathcal{AP}$: We produce T_φ such that $T_\varphi.\tau \in [t_R, t_R]$ using model prediction or real sensor data, based on $\pi[0, t_R]$. We know $\varphi.wpd = 0$ and $\varphi.bpd = 0$ (definition 12). By substitution, we get $T_\varphi.\tau \in [t_R - \varphi.wpd, t_R - \varphi.bpd]$ where $\varphi.wpd = \varphi.bpd = 0$.
- $\varphi = (\circ \psi)$ where $\circ \in \{\Box_I, \Diamond_I\}$: Tuple T_φ with $T_\varphi.\tau = i$ depends on T_ψ for $T_\psi.\tau \in [i + \varphi.lb, i + \varphi.ub]$. To conclude T_φ with $T_\varphi.\tau = i$ at the earliest we require T_ψ with $T_\psi.\tau = i + \varphi.lb$. To produce T_φ with $T_\varphi.\tau = i$ at the latest we require T_ψ with $T_\psi.\tau = i + \varphi.ub$. Correspondingly, with a new computation $\pi[t_R]$, if we produce T_ψ with $T_\psi.\tau = i$, we can potentially produce T_φ with $T_\varphi.\tau \in [i - \varphi.ub, i - \varphi.lb]$. Using induction we have $i \in [t_R - \psi.wpd, t_R - \psi.bpd]$. Therefore, $i \in [t_R - \psi.wpd - \varphi.ub, t_R - \psi.bpd - \varphi.lb]$, and thus $\varphi.wpd = \psi.wpd + \varphi.ub$ and $\varphi.bpd = \psi.bpd + \varphi.lb$.
- $\varphi = \neg\psi$: Special case of $\Box_I$ and $\Diamond_I$ with $I = [0, 0]$.
- $\varphi = (\varphi_1 \circ \varphi_2)$ where $\circ \in \{\mathcal{U}_I, \mathcal{R}_I\}$: For MLTL binary operators, $I = [\varphi.lb, \varphi.ub]$, to produce T_φ with $T_\varphi.\tau = i$, a sufficient condition is based on $\pi[0, t_R]$, we know both T_{φ_1} and T_{φ_2} , where $T_{\varphi_1}.\tau, T_{\varphi_2}.\tau \geq i + \varphi.ub$. We know $T_{\varphi_1}.\tau \geq t_R - \varphi_1.wpd$ using induction. For both inequality conditions for T_{φ_1} , we have $t_R - \varphi_1.wpd \geq i + \varphi.ub$, and $t \leq t_R - \varphi_1.wpd - \varphi.ub$. Similarly, for T_{φ_2} , we have $i \leq t_R - \varphi_2.wpd - \varphi.ub$. By combining the two, we have $i \leq t_R - \max(\varphi_1.wpd, \varphi_2.wpd) - \varphi.ub$. Therefore, $\varphi.wpd = \max(\varphi_1.wpd, \varphi_2.wpd) + \varphi.ub$. Then we check the necessary condition to produce T_φ with $T_\varphi.\tau = i$. The necessary condition is that, based on computation $\pi[0, t_R]$, we know either T_{φ_1} with $T_{\varphi_1}.\tau \geq i + \varphi.lb$, or T_{φ_2} with $T_{\varphi_2}.\tau \geq i + \varphi.lb$. Using presumption, we know $T_{\varphi_1}.\tau \leq t_R - \varphi_1.bpd$. For T_{φ_1} to exist, we have $i + \varphi.lb \leq t_R - \varphi_1.bpd$. Similarly for T_{φ_2} , we have $i + \varphi.lb \leq t_R - \varphi_2.bpd$. To meet with either the condition, we get $i \leq t_R - \min(\varphi_1.bpd, \varphi_2.bpd) - \varphi.lb$. That is to say $\varphi.bpd = \min(\varphi_1.bpd, \varphi_2.bpd) - \varphi.lb$.
- $h = (\varphi_1 \circ \psi)$ where $\circ \in \{\wedge, \vee\}$: Special case of $\mathcal{U}_I$ and $\mathcal{R}_I$ with $I = [0, 0]$. $\square$

Lemma 3. Given an MLTL formula φ and trace $\pi[0, t_R]$, the maximum prediction horizon $\max(H_p)$ required to determine T_φ with $T_\varphi.\tau \geq i$ in at most $i + d$ time stamps (i.e., the satisfiability of φ on π at $i \geq 0$ can be determined within deadline d) is bounded such that $\max(H_p) = \varphi.wpd - d$.

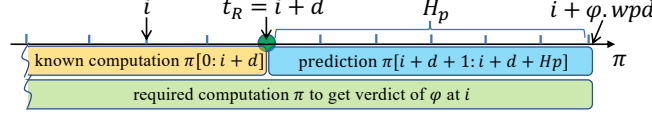


Fig. 12. Pictorial representation of Lemma 3. Note that in this case $t_R = i + d$. Blue part is the trace to predict and yellow part is the known trace.

Proof. By Lemma 1, based on $\pi[0, t_R]$, T_φ is guaranteed to have been produced s.t. $T_\varphi.\tau \in [0, t_R - \varphi.wpd]$. The difference between $i + d$ to $i + \varphi.wpd$ is $\varphi.wpd - d$ events in the computation π . Thus we need to verify over a prediction step of $\varphi.wpd - d$ steps to guarantee T_φ with $T_\varphi.\tau \geq i$ is produced. Therefore, $H_p \leq \varphi.wpd - d$ with $\max(H_p) = \varphi.wpd - d$.

B Hardware Implementation of MPRV

B.1 Shared Connection Queue (SCQ)

In Section 2.2, an MLTL formula is represented as an abstract syntax tree (AST), where sub-formulas are depicted as nodes. To verify an MLTL formula at run-time, we encode each node as an asynchronous observer. We employ queuing structures that we refer to as Shared Connection Queues (SCQs) to buffer and communicate results produced by child nodes for consumption by their parents. Unlike a traditional First In, First Out (FIFO) structure, which has one read pointer and one write pointer, our SCQ is a circular buffer consisting of one write pointer and one or more read pointers. This is a memory-efficient structure for implementing AST topologies that result from performing Common Subexpression Elimination (CSE) on the conical form of an AST, often resulting in multiple parent nodes connecting to a common child. For our use-case, given an MLTL formula, sizing SCQs in accordance to the procedure given in Section 3.2 guarantees SCQs never overflow. A similar data structure, called a Shared Ring Buffer (e.g., [46,25]) has been used in multi-threaded applications.

Fig. 13 illustrates how SCQs are embedded into an MLTL formula's AST. We implement SCQs from Block RAMs (BRAMs), a type of memory resource associated with Field Programmable Gate Arrays (FPGAs). These BRAMs provide a continuous memory space, which is partitioned into multiple SCQs. Each observer node has associated read and write pointers to manage its allocated SCQ memory.

B.2 Architecture Support for MPRV

The operator algorithm logic (Fig. 14) executes the generated assembly-code encoding of an MLTL formula across trace data provided by sensors. The asynchronous observer logic, defined in [37], is synthesized into register-transfer level (RTL) language for computing MLTL operators. Before the RV engine starts, the binary formatted assembly

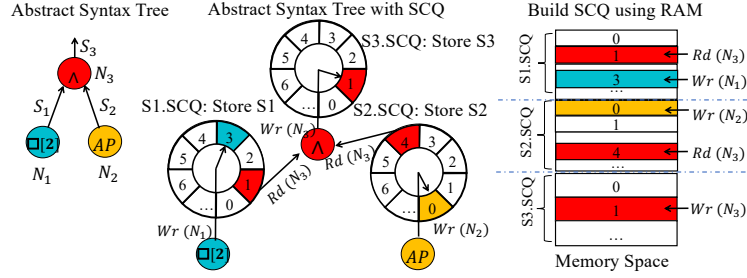


Fig. 13. Implementation of SCQs between observer nodes.

instructions are loaded into the instruction RAM. When the engine starts, a Finite State Machine (FSM) manages the system's operating state by following a transition graph, as seen in Fig. 15. The hardware architecture for MPRV is illustrated in Fig. 14.

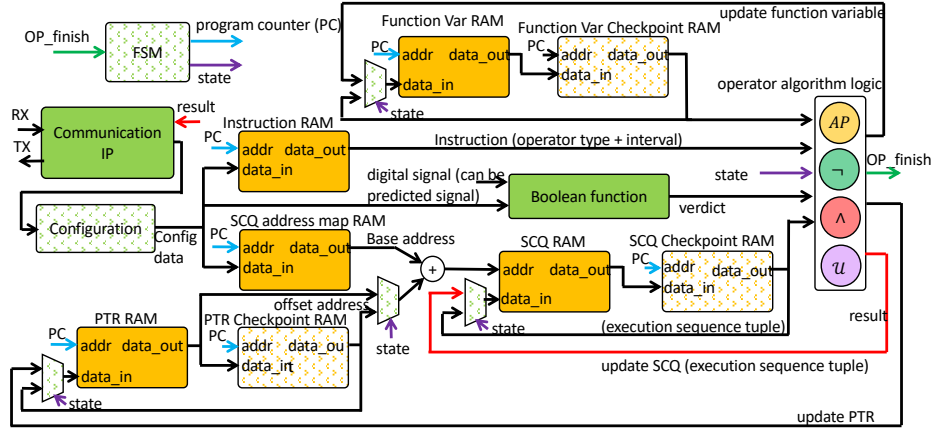


Fig. 14. MPRV augmented RV engine. Components marked with dotted pattern represent new additions or major revisions to R2U2 RV engine to support the predictive RV. For simplification, some arrows are disconnected but color-coded: result is red; state is purple; PC is blue; OP_finish is green. The yellow blocks are preexisting memory components.

The MPRV architecture consists the following primary components:

- **Configuration IP:** Component to communicate with the computer and the FPGA. We use UART in the design.
- **Configuration:** Component to use the pre-defined protocol to convert communication data into the format for the system configuration before the RV engine starts.
- **Instruction RAM:** Memory storage for the binary encoding of assembly instructions. We use the Program Counter (PC) to address instructions.
- **Operator Algorithm Logic:** Synthesized logical block implementing the Algorithms in [37] for MLTL operators, and updates **Function Var** and **PTR** after operator execution.
- **Boolean Function:** An arithmetic block consisting of adders, multipliers, and comparators, which converts signals into $\mathcal{AP}$ s.
- **Function var RAM:** Stores the algorithm's local variables for each instruction.

- **SCQ**: One RAM partitioned into SCQs for observer communication. Refer to *Memory Space* in Fig. 13.
- **SCQ address map RAM**: The base address for each instruction when accessing its **SCQ**. The address is decided by Algorithm 4 that splits the SCQ RAM for each instruction by specifying the address.
- **PTR RAM**: Pointer RAM. Each data content contains one write pointer and two read pointers for each instruction. The sum of the corresponding base address and the pointer is the absolute address (Rd and Wr in Fig. 13) to access the SCQ RAM.
- **Checkpoint RAM**: Memory component for checkpointing data before performing prediction.

For the memory component, the write and read enable signal is controlled by the FSM. The **Instruction RAM** and the **SCQ address map RAM** content is online configurable and is capable of handling different MLTL formulas without resynthesizing the Register-transfer level (RTL) design. Fig. 15 details the hardware’s state transition flow. Starting from idle, the verification process is triggered by the real-time clock (RTC) periodically. First the system conducts RV without prediction by using the available sensor information. The process of RV is marked by shaded blue in Fig. 15. For RV *without* prediction ($\max(H_p) = 0$), the detailed process includes 1. reading digital sensor signals and converting into $\mathcal{AP}$ s; 2. loading instructions; 3. loading the input data from **SCQ**s or directly from $\mathcal{AP}$ s, and fetching corresponding **Function Var** for the specific operator logic; 4. execution; 5. writing back results to **SCQ**, updating **Function Var**, and incrementing the PC. The RV engine outputs a result when PC reaches End of Instruction (**EoI**). For RV *with* prediction, the system will checkpoint the current data content of the **SCQ**, **PTR** and **Function Vars** of the algorithm and conduct sensor value prediction, and repeat normal RV engine process based on the predictive sensor signals for some iterations according to the prediction steps until H_p reaches $\max(H_p)$. After RV with prediction completes, the system state is restored from the **Checkpoint RAM**. The system then waits for new sensor data to arrive and repeat the process.

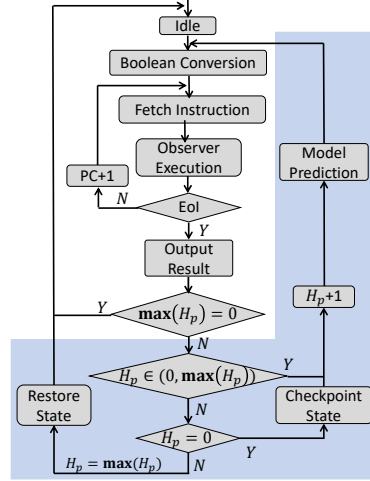


Fig. 15. Finite state machine transitions inside Fig. 14. The transition graph marked with a shaded background is the augmentation of the FSM within the RV engine to support prediction.

C MPRV on the point-mass

For our example system, we have a point-mass of mass m . The objective is to have the point-mass follow a given trajectory to a desired location by applying an external force. The point-mass state-space model contains two state variables, namely its position p and velocity $\dot{p}$. The input is a force f applied to the point-mass. Using Newton’s laws

of motion, we can establish the following formula:

$$\begin{bmatrix} \dot{p} \\ \ddot{p} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \times \begin{bmatrix} p \\ \dot{p} \end{bmatrix} + \begin{bmatrix} 0 \\ 1/m \end{bmatrix} \times [f]$$

We convert this model from continuous-time to discrete-time based on a specific state sampling rate. For this example, a sampling rate of 10Hz is used.

$$A = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}, B = \begin{bmatrix} 0.005 \\ 0.1 \end{bmatrix}, C = [1 \ 0]$$

The input force u_k is a function of the state x_k and control algorithm (e.g., MPC). Additionally, knowing the control algorithm and current state of the model allows us to estimate future states of the system.

We use an MPC controller to guide the point-mass along a predefined trajectory. The key point of MPC is to minimize a cost function subject to constraints. In our simulation, the input force is constrained to the range $[-1N, 1N]$. When minimizing the cost function, we associate a multiplicative cost weighting of 2 with the error in mass position and a weighting of 1 with its speed. The MPC prediction horizon has been set to 100 update steps into the future, and the controller actuation update rate to 10 Hz. The resulting trajectory of the point-mass compared to the planned trajectory is shown in the upper portion of Fig. 16.

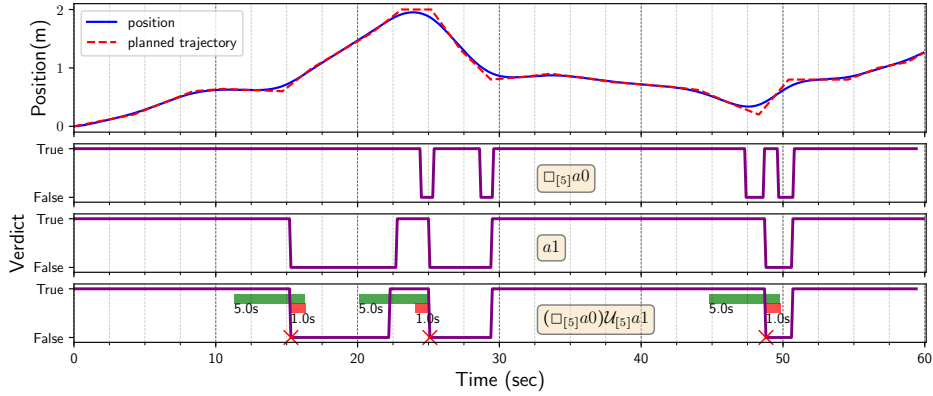


Fig. 16. Model predictive control of the position of a point-mass, and verification results of three formulas. The colored bars in the bottom figure indicate the time when the system detects the error marked by red $\times$. The right end of each bar is the formula-violation detection-time using RV without prediction. The left end of each green (orange) bar is the formula-violation detection-time using MPRV with a prediction horizon of 5s (1s).

Let trace π be a combination of predicates $a0$ and $a1$ at each time stamp. We specify the following Boolean mapping function to convert system states to atomics: $a0$: absolute value of trajectory error $< 0.1m$, and $a1$: absolute speed $< 0.12m/s$. These atomics are used within three MLTL formulas that are specified and evaluated in Fig. 16. For formula φ in each figure, if the verdict is true, it means that $\pi, t \models \varphi$, where t is the corresponding x-axis value. Due to the nature of future time asynchronous observers, we

may not know the valuation of $\pi, t \models \varphi$ until a time later than t . In Fig. 16, using colored bars, we show the "real-time" at which we are able to know (estimate) a formula becoming **false**. This figure also compares the responsiveness of formula $(\Box_{[5]} a0)\mathcal{U}_{[5]} a1$ ³ based on the deadline $d=90$ steps (9s) and $d=50$ steps (5s), which results in the max prediction horizon of 10 steps (1s) and 50 steps (5s) into the future according to Lemma 3. The green bar is associated with a 50 step prediction horizon. The left boundary of the green bar is the time when MPRV detects the formula becoming **false**, and right boundary is the time when a **false** valuation is detected without prediction. Similarly for a 10 step prediction horizon, the orange bar depicts this behavior.

³ Time intervals in the formula are converted into seconds to match with the graph. The real formula is $(\Box_{[50]} a0)\mathcal{U}_{[50]} a1$ under 10Hz sample rate.